

Algoritmos de Engenharia

Programa de Pós-Graduação e Engenharia Elétrica e de Computação

Centro de Tecnologia – UFRN

ORDER STATISTICS

- *i th order statistic* is the i th smallest element of a set of n elements.
- The *minimum* is the first order statistic ($i = 1$).
- The *maximum* is the n th order statistic ($i = n$).
- A *median* is the “halfway point” of the set.
- When n is odd, the median is unique, at $i = (n + 1)/2$.
- When n is even, there are two medians:
 - The *lower median*, at $i = n/2$, and
 - The *upper median*, at $i = n/2 + 1$.
 - We mean lower median when we use the phrase “the median.”

ORDER STATISTICS

The *selection problem*:

Input: A set A of n distinct numbers and a number i , with $1 \leq i \leq n$.

Output: The element $x \in A$ that is larger than exactly $i - 1$ other elements in A .
In other words, the i th smallest element of A .

Easy to solve the selection problem in $O(n \lg n)$ time:

- Sort the numbers using an $O(n \lg n)$ -time algorithm, such as heapsort or merge sort.
- Then return the i th element in the sorted array.

MINIMUM AND MAXIMUM

We can easily obtain an upper bound of $n - 1$ comparisons for finding the minimum of a set of n elements.

- Examine each element in turn and keep track of the smallest one.
- This is the best we can do, because each element, except the minimum, must be compared to a smaller element at least once.

The following pseudocode finds the minimum element in array $A[1:n]$:

MINIMUM(A, n)

```
1  min =  $A[1]$ 
2  for  $i = 2$  to  $n$ 
3      if  $\textit{min} > A[i]$ 
4           $\textit{min} = A[i]$ 
5  return min
```

The maximum can be found in exactly the same way by replacing the $>$ with $<$ in the above algorithm.

SIMULTANEOUS MINIMUM AND MAXIMUM

Some applications need both the minimum and maximum of a set of elements.

- For example, a graphics program may need to scale a set of (x, y) data to fit onto a rectangular display. To do so, the program must first find the minimum and maximum of each coordinate.

A simple algorithm to find the minimum and maximum is to find each one independently. There will be $n - 1$ comparisons for the minimum and $n - 1$ comparisons for the maximum, for a total of $2n - 2$ comparisons. This will result in $\Theta(n)$ time.

SIMULTANEOUS MINIMUM AND MAXIMUM

(continued)

In fact, at most $3\lfloor n/2 \rfloor$ comparisons suffice to find both the minimum and maximum:

- Maintain the minimum and maximum of elements seen so far.
- Don't compare each element to the minimum and maximum separately.
- Process elements in pairs.
- Compare the elements of a pair to each other.
- Then compare the larger element to the maximum so far, and compare the smaller element to the minimum so far.

This leads to only 3 comparisons for every 2 elements.

SIMULTANEOUS MINIMUM AND MAXIMUM

(continued)

Setting up the initial values for the min and max depends on whether n is odd or even.

- If n is even, compare the first two elements and assign the larger to max and the smaller to min. Then process the rest of the elements in pairs.
- If n is odd, set both min and max to the first element. Then process the rest of the elements in pairs.

SELECTION IN EXPECTED LINEAR TIME

Selection of the i th smallest element of the array A can be done in $\Theta(n)$ time.

The function RANDOMIZED-SELECT uses RANDOMIZED-PARTITION from the quicksort algorithm in Chapter 7. RANDOMIZED-SELECT differs from quicksort because it recurses on one side of the partition only.

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$           //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$           // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```


SELECTION IN EXPECTED LINEAR TIME (continued)

After the call to RANDOMIZED-PARTITION, the array is partitioned into two subarrays $A[p : q - 1]$ and $A[q + 1 : r]$, along with a **pivot** element $A[q]$.

- The elements of subarray $A[p : q - 1]$ are all $\leq A[q]$.
- The elements of subarray $A[q + 1 : r]$ are all $> A[q]$.
- The pivot element is the k th element of the subarray $A[p : r]$, where $k = q - p + 1$.
- If the pivot element is the i th smallest element (i.e., $i = k$), return $A[q]$.
- Otherwise, recurse on the subarray containing the i th smallest element.
 - If $i < k$, this subarray is $A[p : q - 1]$, and we want the i th smallest element.
 - If $i > k$, this subarray is $A[q + 1 : r]$ and, since there are k elements in $A[p : r]$ that precede $A[q + 1 : r]$, we want the $(i - k)$ th smallest element of this subarray.

Exercises

1. Implemente o algoritmo de mínimo e máximo simultâneos da seção 9.1 do livro do Cormen, 4a Ed na sua linguagem favorita e mostre através de medição de tempo que é mais rápido que a abordagem não-simultânea para um vetor de entrada suficientemente grande.
2. Implemente os algoritmos de seleção aleatória e seleção das seções 9.2 e 9.3 do livro do Cormen, 4a Ed., e realize experimentos numéricos para demonstrar em quais casos um tem vantagens com relação ao outro.
3. Implemente o algoritmo da mediana ponderada e use-o para resolver o item e do Problema 9-3 do Cormen, 4a Ed.